Дискретный анализ

Вайб-лекция 5: поиск подстроки

Андрей Калинин

Дискретный анализ, 2026/27

14 октября 2026

Материалы лекций

Материалы курса, регистрация на конкурс и вопросы в Issues:

`https://github.com/alk-2000/da-lectures`



Конкурс по сортировке

Задача: самая быстрая сортировка 32-битных беззнаковых чисел.

Проверяющая программа публикуется в репозитории курса; следите за Issues.

Срок сдачи: 6 декабря, 23:59 МСК.

Первое место — «отлично»; ещё четыре места — +1 балл на экзамене. Победители объясняют решения на семинаре.



Как записаться на конкурс

- В папке `2026/contest/sorting/participants` создайте файл `<ник>.md` с вашим GitHub-аккаунтом.
- Нажмите **Propose new file**, затем **Create pull request**.
- На **andrey@kalinin.ru**: тема «Конкурс по сортировке», ФИО, группа, GitHub-аккаунт.
- Решение до срока хранится в закрытом репозитории. Публично его не выкладываем.

Сегодня: поиск = фильтр + точная проверка

Почти любой быстрый поиск устроен одинаково: **дешёвый фильтр** отбрасывает большинство кандидатов, **точная проверка** подтверждает остальных.

Два независимых вопроса решают скорость:

1. **Окупается ли фильтр?** Он тоже стоит времени, а «нет» должно быть частым.
2. **Не сравнивает ли точная проверка одно и то же дважды?**

План: фильтр Блума → ошибка в CPython → Z, КМП, Бойер–Мур, Апостолико–Джанкарло → как CPython это починил → SIMD → Ахо–Корасик → суффиксные деревья.

Почему DuckDB не заменит ART фильтром Блума?

Фильтр Блума занимает в десятки раз меньше памяти, чем ART. Что мешает? Сначала — что должен уметь индекс:

Запрос	Что должен вернуть индекс
`key = "cat"`	номер строки таблицы, `row_id = 8`
проверка `UNIQUE`	точно: такой ключ уже есть или нет
`key BETWEEN a AND b`	все ключи диапазона по порядку

DuckDB: Persistent Storage of Adaptive Radix Trees, 2022

Битовый массив — вся память фильтра



Фильтр — это *b* бит, вначале все нули. Ключи в нём *не хранятся*.

Хеш ключа выбирает *k* адресов. При вставке по этим адресам пишем 1. Один массив — на всё множество ключей.

Как из cat получить адреса



Адрес = число по модулю $b = 16$. Для того же ключа адреса всегда те же.

Вычислено: demo/illustrations.py; numbers.md, N08.

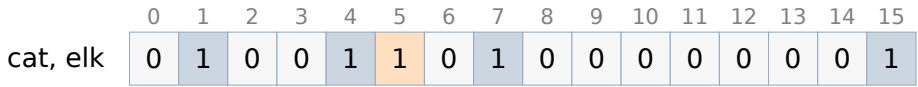
Вставка: поставить выбранные биты

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
cat	0	1	0	0	0	1	0	1	0	0	0	0	0	0	0	0

Адреса для cat: $7, 1, 5$. Ставим три единицы.

Повторная вставка ничего не меняет: 1 остаётся 1.

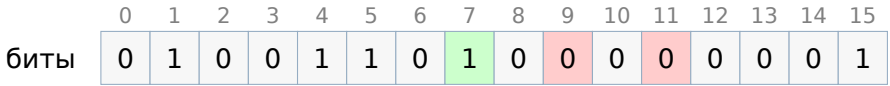
Два ключа могут делить один бит



Адреса для `elk`: $15, 4, 5$. Бит 5 уже поставил `cat`.

В массиве нет записи «этот бит принадлежит elk». Всего единиц пять, а не шесть.

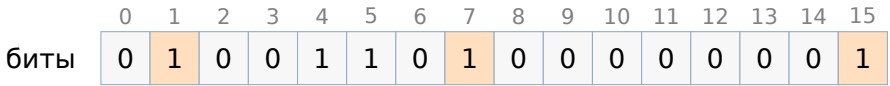
Один ноль — ключа точно нет



Адреса для fox: $7, 11, 9$. В позиции 11 стоит 0.

Если бы fox добавляли, все три бита были бы единицами. Значит, не добавляли.

Все единицы — ключ, возможно, есть



Адреса для `tiger`: $1, 15, 7$. Все три бита равны 1.

Но мы вставляли только `cat` и `elk`. Ответ «возможно» требует **точной проверки**.

Фильтр Блума: весь алгоритм

```
add(key):
    for p in positions(key):
        bits[p] = 1

may_contain(key):
    for p in positions(key):
        if bits[p] == 0:
            return false
    return true
```

- `positions(key)` — те же k адресов при вставке и при запросе.
- `true` — разрешение проверить точно, не доказательство.
- Время запроса: один хеш и до k чтений бита.

Учебный псевдокод; индексы с нуля. Burton Bloom, *Space/time trade-offs in hash coding with allowable errors*, CACM 1970.

Вопрос залу

Можно ли удалить ключ, обнулив его биты?

Подумайте на примере: в фильтре `cat` $(7, 1, 5)$ и `elk` $(15, 4, 5)$. Удаляем `elk`.

Ответ: нельзя — бит 5 общий

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
до	0	1	0	0	1	1	0	1	0	0	0	0	0	0	0	1
стерли elk	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0

Стирая биты `elk` ($15, 4, 5$), мы стираем и бит 5, который нужен `cat`. После этого фильтр ответит «`cat` точно нет» — ложноотрицательно.

Выход: хранить не биты, а счётчики (counting Bloom filter) — в несколько раз больше памяти.

Что умеет обычный фильтр Блума

Операция	Обычный фильтр Блума
Добавить ключ	да
Проверить ключ	«точно нет» или «возможно»
Удалить ключ	нельзя
Объединить два фильтра	OR битов при одинаковых параметрах
Сохранить на диск	биты + параметры + способ кодирования ключей
Получить ключ, `row_id`, порядок	**этих данных внутри нет**

Четыре числа: N, b, k, f

Символ	Что считаем	Наш пример
N	разных добавленных ключей	10 000
b	битов во всём массиве	100 000
k	адресов на один ключ	7
f	доля ложных «возможно» среди отсутствующих	$\approx 0{,}0082$

«10 бит на ключ» — это отношение b/N, а не десять личных ячеек: массив общий, $100\,000$ бит $= 12\,500$ байт.

Откуда берётся вероятность ошибки

N ключей по k адресов — всего kN записей единиц в массив из b бит.

Одна запись не трогает данный бит с вероятностью $1 - 1/b$.

После kN записей бит остался нулём с вероятностью $(1 - 1/b)^{kN} \approx e^{-kN/b}$.

У отсутствующего ключа все k адресов должны оказаться единицами:

$$f \approx \left(1 - e^{-kN/b}\right)^k.$$

Модель равномерного независимого хеширования; формула приближённая. Bloom, CACM 1970; Broder, Mitzenmacher, *Network Applications of Bloom Filters*, 2004.

Сколько адресов на ключ, и что будет при росте

При $b/N = 10$ ошибка минимальна при $k \approx (b/N)\ln 2 \approx 6{,}9$. Берём 7: доля единиц $\approx 50{,}3\%$, $f \approx 0{,}503^7 \approx 0{,}82\%$.

Те же $b = 100\,000$ и $k = 7$, но ключей больше:

Ключей N	Единиц в массиве	Ложных «возможно»
10 000	50,3%	0,82%
20 000	75,3%	13,8%
50 000	97,0%	80,7%
100 000	99,9%	99,4%

Расчёт по формуле: demo/illustrations.py; numbers.md, N08. Размер задаётся заранее: $b \approx -N\ln f/(\ln 2)^2$; вырос набор — фильтр строят заново из ключей.

Проверим на настоящих запросах

Что сделали	Результат
Добавили разные ключи	строки «0» … «9999»
Массив	100 000 бит, $k = 7$
Проверили отсутствующие ключи	100 000 запросов: «10000» … «109999»
Ложных «возможно»	775 из 100 000 $= 0{,}775\%$
Проверили все добавленные ключи	0 пропусков

Формула обещала $0{,}82\%$. Ложноотрицательных нет, как и доказано.

Наш запуск 10.10.2026; demo/bloom_probe.py, demo/bloom-results.json; numbers.md, N03.

Фильтр должен окупить собственную работу

q — доля запросов, для которых ключа нет; f — доля ложных «возможно».

C_f — цена проверки фильтром; C_e — цена точной проверки.

$$C_{\text{с фильтром}} = C_f + \big(1 - q + qf\big)\, C_e \qquad \text{выигрыш, когда} \qquad C_f < q(1-f)\, C_e.$$

Фильтр полезен, когда **«нет» частое** и **точная проверка дорогая** (чтение с диска), и бесполезен, когда почти все ключи есть.

Это формула всего дня. Она вернётся в CPython и в SIMD.

LevelDB: где фильтр окупается

LevelDB — встраиваемое хранилище «ключ → значение» от Google (2011), предок RocksDB. Операции: `Put`, `Get`, `Delete`.

На диске ключи лежат упорядоченно в файлах SSTable, разбитых на блоки. Для группы блоков хранится свой фильтр Блума.

При `Get(key)` фильтр говорит «в этих блоках ключа точно нет» — и блок с диска не читается. Это именно случай $C_e \gg C_f$.

Дальше — три фрагмента файла `util/bloom.cc`, версия 1.23, лицензия BSD. Слева номера строк оригинала.

LevelDB: SSTable и filter block; util/bloom.cc 1.23

LevelDB: число проб — инженерный выбор

```
19  explicit BloomFilterPolicy(int bits_per_key) :
      bits_per_key_(bits_per_key) {
20    // We intentionally round down to reduce
      probing cost a little bit
21    k_ = static_cast<size_t>(bits_per_key * 0.69);
       // 0.69 =~ ln(2)
22    if (k_ < 1) k_ = 1;
23    if (k_ > 30) k_ = 30;
24  }
```

google/leveldb 1.23 · BSD-3-Clause leveldb/util/bloom.cc:19–24

- Строка 21: $k = \lfloor 0{,}69 \cdot b/N \rfloor$ — наша формула $(b/N)\ln 2$.
- Строка 20: округляют *вниз* намеренно: каждая проба — чтение памяти.
- При 10 битах на ключ $k = 6$, а не 7: чуть больше ошибок, меньше работы.

LevelDB: k адресов из одного хеша

```
43    for (int i = 0; i < n; i++) {
44      // Use double-hashing to generate a sequence
      of hash values.
45      // See analysis in [Kirsch,Mitzenmacher
      2006].
46      uint32_t h = BloomHash(keys[i]);
47      const uint32_t delta = (h >> 17) | (h <<
      15);  // Rotate right 17 bits
48      for (size_t j = 0; j < k_; j++) {
49        const uint32_t bitpos = h % bits;
50        array[bitpos / 8] |= (1 << (bitpos % 8));
51        h += delta;
52      }
53    }
```

- Строка 46: *один* 32-битный хеш на ключ, не k хешей.
- Строки 47, 51: следующий адрес $= h +$ delta, где `delta` — тот же хеш, повёрнутый на 17 бит («double hashing»).
- Строка 50: `bitpos/8` — байт, `bitpos%8` — бит в нём.

google/leveldb 1.23 · BSD-3-Clause leveldb/util/bloom.cc:43–53; Kirsch, Mitzenmacher, *Less Hashing, Same Performance*, 2006

LevelDB: запрос читает те же биты

```
72    uint32_t h = BloomHash(key);
73    const uint32_t delta = (h >> 17) | (h << 15);
      // Rotate right 17 bits
74    for (size_t j = 0; j < k; j++) {
75      const uint32_t bitpos = h % bits;
76      if ((array[bitpos / 8] & (1 << (bitpos %
      8))) == 0) return false;
77      h += delta;
78    }
79    return true;
```

- Строки 72–73: тот же хеш и та же `delta`, что при вставке.
- Строка 76: первый нулевой бит — `false`, «точно нет».
- Строка 79: `true` — «может быть». Функция так и называется: `KeyMayMatch`.

google/leveldb 1.23 · BSD-3-Clause leveldb/util/bloom.cc:72–79

Ответ: почему фильтр не заменяет ART

Что нужно индексу	Чего нет у фильтра Блума
Найти строку по ключу	нет `row_id`
Точно проверить `UNIQUE`	«возможно» не окончательно
Проверить внешний ключ	нужно подтвердить существование
Прочитать диапазон	нет порядка ключей
Удалить ключ	биты общие

Фильтр может стоять *перед* точным индексом и экономить обращения к нему. Заменить индекс он не может: после «возможно» всё равно нужна точная проверка.

DuckDB: сохранение ART, 2022

Где DuckDB действительно использует фильтр Блума

дополнительно

В файлах Parquet для столбца внутри группы строк можно хранить фильтр Блума.

Запрос `column = 42`: фильтр говорит «точно нет» — группу строк не читаем вовсе. «Возможно» — читаем данные и проверяем условие точно.

DuckDB читает и пишет такие фильтры с версии 1.2 (2025). Это другая подсистема, чем сохранение ART (2022).

DuckDB: Parquet Bloom Filters, 2025; спецификация Parquet

А если положить в фильтр триграммы текста?

Хотим искать подстроку abcd по множеству файлов. Для каждого файла — фильтр из всех троек подряд идущих букв.

Запрос abcd требует триграмм abc и bcd. Файл abcXbcd содержит обе — и не содержит abcd.

Даже *точное* множество триграмм даёт только **кандидата**. Фильтр Блума добавляет к этому свои ложные ответы. Дальше всё равно нужен **точный поиск подстроки** — и с него начинается остальной день.

Russ Cox: Regular Expression Matching with a Trigram Index (Google Code Search)

Почему `bytes.find` внезапно стал медленным?

CPython, октябрь 2020. Пользователь ищет в большом файле длинную серию одинаковых байтов. Образец длины $m - 1$ находится мгновенно. Образец длины m «зависает».

У `find` *есть* фильтр. Сейчас увидим, что фильтр исправен — и это не помогает.

CPython: issue 86138 (bpo-41972)

Воспроизводитель: два почти одинаковых запроса

```
BIG = 24000
text = bytearray(b'A' * BIG + b'B' * BIG)
third = BIG // 3
text[third:2*third] = b'B' * third

short_pattern = b'B' * (BIG - 1)
long_pattern  = b'B' * BIG
# Both answers: BIG
```

- Текст: серия A, серия B, внутри серии A — вставка B.
- Два образца из одних B; отличаются на один байт.
- Оба поиска должны вернуть позицию 24 000.

Уменьшенный вариант публичного воспроизводителя: reproducer.py из обсуждения.

Наш запуск: лишний байт меняет время в сотни раз

Длина образца	CPython 3.9.0	CPython 3.10.0
23 999 байт	11,8 мкс	11,6 мкс
24 000 байт	**7 960 мкс**	79 мкс

Текст 48 000 байт, ответ в обоих случаях 24 000. Медианы пяти серий; для 3.9.0 на длинном образце разброс серий 6,9–12,6 мс, медиана в 677 раз больше, чем на коротком.

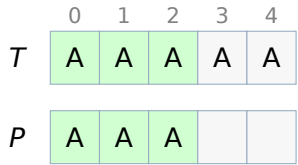
В 3.10 ошибку починили. Чтобы понять как — сначала разберём, что именно делал поиск в 3.9.

Наш запуск 10.10.2026, C-ядро fastsearch из двух версий CPython, GCC 13.3 -O3, Ryzen AI MAX+ 395, CPU 1; demo/fastsearch-results.csv; numbers.md, N01.

Сначала договоримся, что ищем

T — текст длины n, P — образец длины m. Сравниваем **байты**.

Вхождение в позиции s: $T[s..s+m) = P$. Индексы с нуля, правая граница не включена.



Первое вхождение: 0. Все вхождения: $\{0, 1, 2\}$ — **перекрытия тоже считаются**.

`find` возвращает первое. Наши учебные алгоритмы и замеры ниже перечисляют все.

Наивный поиск: проверить каждую позицию

```
find_all(T, P):
    for s = 0 .. n−m:
        j = 0
        while j < m and T[s+j] == P[j]:
            j += 1
        if j == m:
            emit(s)
```

- Для каждого начала s сравниваем слева направо до первого различия.
- Время $O((n - m + 1)\, m)$ в худшем случае; память $O(1)$.
- На случайном тексте различие обычно в первой-второй букве, и это быстро.

Учебный псевдокод; индексы с нуля.

Тот же наивный поиск: наш C++ и ядро Linux дополнительно

Наш (demo/search_bench.cpp:19–27), считает все вхождения:

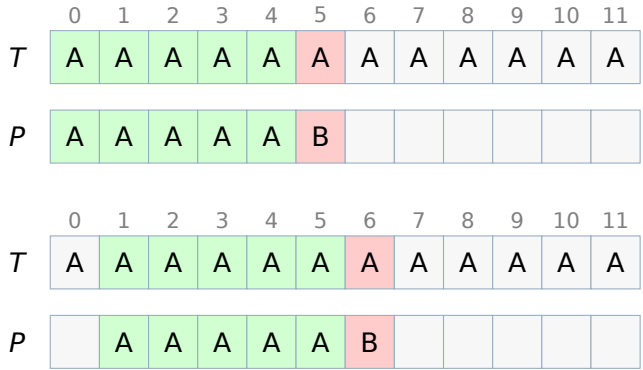
```cpp
int naive(unsigned char* p,int m,unsigned
     char* t,int n) {
    int count=0;
    for(int i=0;i<=n-m;++i) {
        int j=0;
        while(j<m && t[i+j]==p[j]) ++j;
        count+=j==m;
    }
    return count;
}
```

Linux, lib/string.c:737–748 — универсальный `strstr`:

```cpp
  l2 = strlen(s2);
  if (!l2)
    return (char *)s1;
  l1 = strlen(s1);
  while (l1 >= l2) {
    l1--;
    if (!memcmp(s1, s2, l2))
      return (char *)s1;
    s1++;
  }
  return NULL;
}
```

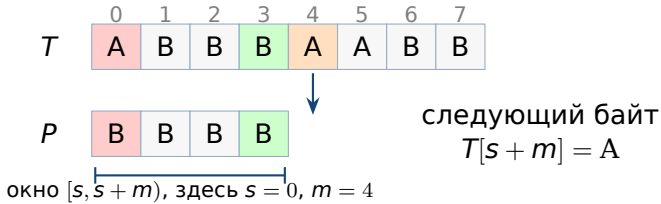
Наш код подготовлен с помощью ИИ. Linux v6.12 · GPL-2.0: lib/string.c; используется, когда у архитектуры нет своего strstr.

На повторах мы сравниваем одно и то же



Каждый сдвиг на 1 заново сравнивает те же пять A. Для образца длины m — почти m сравнений на позицию.

Как CPython 3.9 проверял очередное окно



1. Сравнить **последний** байт образца с последним байтом окна.
2. Совпал — сравнить остальные слева направо.
3. Не совпало — посмотреть на **байт сразу после окна**: есть ли он вообще в образце?

CPython 3.9.0: цикл поиска в fastsearch.h; тот же цикл сохранён в 3.10 как «default_find»

Маска символов: фильтр Блума на одно число

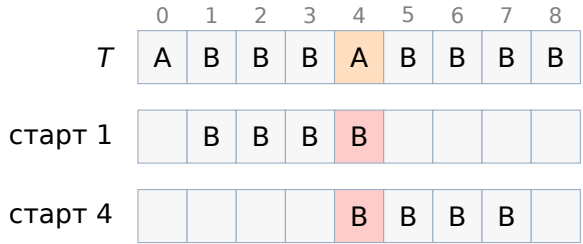
```
36 #define STRINGLIB_BLOOM_ADD(mask, ch) \
37     ((mask |= (1UL << ((ch) &
       (STRINGLIB_BLOOM_WIDTH -1)))))
38 #define STRINGLIB_BLOOM(mask, ch)     \
39     ((mask &  (1UL << ((ch) &
       (STRINGLIB_BLOOM_WIDTH -1)))))
```

mask — одно 64-битное число. Для каждого байта *c* образца ставим бит номер $c \bmod 64$.

- Макрос так и называется: BLOOM. Это фильтр Блума с $b = 64$, $k = 1$.
- B — код 66, бит 2. A — код 65, бит 1.
- Образец BBBB: установлен только бит 2. Запрос A: бит 1 равен 0 — «точно нет».

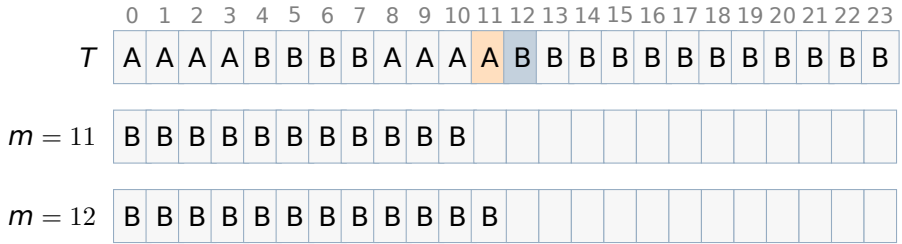
python/cpython 3.10.0 · PSF fastsearch.h:36–39 (в 3.9.0 те же макросы)

Почему байт после окна разрешает большой сдвиг



Образец BBBB не содержит A. Любое начало $1, 2, 3, 4$ накроет эту A — значит, все четыре невозможны. После неудачи в начале 0 переходим сразу к началу 5.

Тот же текст, другой байт после окна



Образец на один байт длиннее — и после окна стоит B, а не A. B *есть* в образце: фильтр честно отвечает «возможно», большой сдвиг запрещён.

Фильтр отвечает правильно. Куда уходят почти 8 миллисекунд?

Текст: 24 000 A, 24 000 B. Образец: 24 000 B. Вспомните, как проверяется окно после того, как фильтр сказал «возможно».

Ответ: точная проверка повторяет сравнения

На каждом окне внутри серии B последний байт совпадает, и проверка идёт слева направо до первой A: тысячи сравнений на окно, тысячи окон. Счёт по модели цикла 3.9: короткий образец — 2 окна и 24 000 сравнений; длинный — 24 001 окно и **32 075 999 сравнений**.

Фильтр отвечает на *свой* вопрос: «есть ли символ в образце?» Он не помнит, сколько символов мы уже сравнили в соседнем окне.

Нужна информация о **совпавших частях строк**. Четыре классических алгоритма — четыре способа её хранить:

Z-функция	длины совпадений с началом строки
Кнут–Моррис–Пратт	границы префиксов образца; текст читается один раз
Бойер–Мур	сравнение справа налево, сдвиг по двум правилам
Апостолико–Джанкарло	Бойер–Мур, который помнит, что уже сравнил

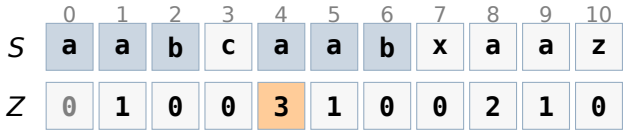
Как далеко строка совпадает со своим сдвигом?

Первый способ хранить информацию о совпавших частях: для каждой позиции i записать, сколько символов подряд совпадает у $S[i..)$ с началом S.

Сначала определение и картинка, затем линейный алгоритм с доказательством, затем настоящий код.

Gusfield, *Algorithms on Strings, Trees, and Sequences*, 1997, гл. 1–2; лекция alk 08.10.2010 (legacy/2010/lectures/20101008/zblocks.tex).

$Z[i]$ — длина совпадения $S[i..)$ с началом строки



$Z[4] = 3$: $S[4..7) =$ aab $= S[0..3)$, а дальше $S[7] =$ x $\neq S[3] =$ c.

$Z[8] = 2$: aa, затем z против b. $Z[1] = 1$: a, затем a против b.

$Z[0]$ не нужен: вся строка совпадает сама с собой. Кладём 0 по соглашению.

Из Z-функции сразу получается поиск

Склеим $S = P \# T$, где # не встречается ни в P, ни в T:

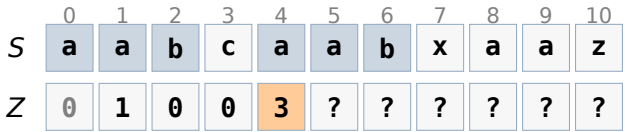
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
S	a	b	a	a	b	#	a	b	a	a	b	a	a	b	a	c	b
Z	0	0	1	2	0	0	5	0	1	5	0	1	3	0	1	0	0

$Z[i] \geq m$ ровно там, где в S с позиции i начинается копия P. Разделитель не даёт совпадению перейти через границу, поэтому $Z[i] \leq m$.

Текст начинается в S с индекса $m + 1 = 6$: вхождения $i - 6$, то есть 0 и 3. Перекрывающиеся вхождения находятся сами собой.

Вопрос залу

Мы знаем $Z[4] = 3$. Что известно про $Z[5]$ и $Z[6]$, не глядя в строку?



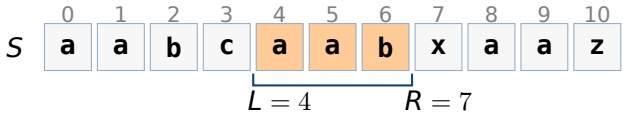
Считать каждый $Z[i]$ заново — на строке a^n это $(n-1)+(n-2)+\cdots+1$ сравнений.

Ответ: внутри блока ответ уже посчитан

$S[4..7) = S[0..3)$, поэтому $S[5..7) = S[1..3)$ и $S[6..7) = S[2..3)$.

Как $S[1..)$ совпадает с началом, мы уже знаем: $Z[1] = 1$. Совпадение и различие ($S[2] \neq S[1]$) лежат *внутри* скопированного куска, значит $Z[5] = Z[1] = 1$. Аналогично $Z[6] = Z[2] = 0$. Ни одного нового сравнения.

Нужно только помнить **самый правый** уже найденный кусок, совпадающий с началом строки.



Z-блок $[L, R)$: $S[L..R) = S[0..R-L)$, R — самый правый конец среди уже обработанных позиций.

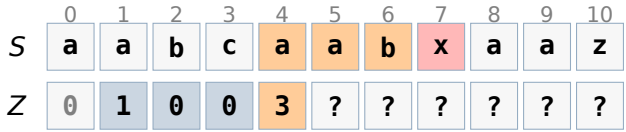
Линейный алгоритм Z

```
1  Z[0 .. n) = 0
2  L = R = 0
3  for i = 1 .. n−1:
4      if i < R:
5          Z[i] = min(R−i, Z[i−L])
6      while i+Z[i] < n and S[Z[i]] == S[i+Z[i]]:
7          Z[i] += 1
8      if i+Z[i] > R:
9          L = i
10         R = i+Z[i]
```

- Строки 4–5: если i внутри блока, начинаем с готового $Z[i-L]$, но не дальше R.
- Строки 6–7: сравниваем дальше, пока совпадает.
- Строки 8–10: нашли более правый конец — запоминаем новый блок.
- Три случая: $i \geq R$; $Z[i-L] < R-i$ — копируем; $Z[i-L] \geq R-i$ — досравниваем от R.

Учебный псевдокод; индексы с нуля, $[L, R)$ не включает R. Соответствует Gusfield, алг. 1.4.

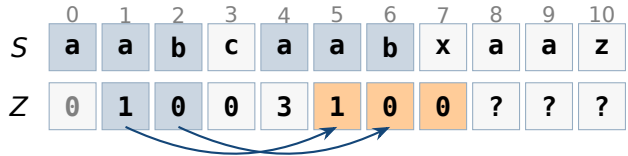
Трасса, шаги $i = 1 \dots 4$: все вне блока



i	случай	сравнения	$Z[i]$	$[L, R)$ после
1	$i \geq R$	$S[0] = S[1]$, $S[1] \neq S[2]$	1	$[1, 2)$
2	$i \geq R$	$S[0] \neq S[2]$	0	$[1, 2)$
3	$i \geq R$	$S[0] \neq S[3]$	0	$[3, 3)$
4	$i \geq R$	$S[0..3) = S[4..7)$, $S[3] \neq S[7]$	3	$[4, 7)$

Пока $i \geq R$, ничего не известно: сравниваем честно. Шаг 4 нашёл блок $[4, 7)$.

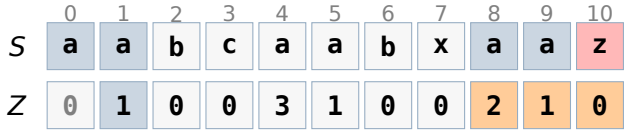
Трасса, шаги $i = 5, 6, 7$: копируем ответ



i	случай	откуда	$Z[i]$	$[L, R)$
5	$i < R$, $Z[i-L] = Z[1] = 1 < R - i = 2$	копия $Z[1]$	1	$[4, 7)$
6	$i < R$, $Z[2] = 0 < R - i = 1$	копия $Z[2]$	0	$[4, 7)$
7	$i \geq R$	$S[0] \neq S[7]$	0	$[4, 7)$

Шаги 5 и 6 не читают строку: различие известно заранее, оно внутри блока.

Трасса, шаги $i = 8, 9, 10$: досравниваем от R



i	случай	что делаем	$Z[i]$	$[L, R)$
8	$i \geq R$	$S[0..2) = S[8..10)$, $S[2] \neq S[10]$	2	$[8, 10)$
9	$i < R$, $Z[1] = 1 \geq R - i = 1$	старт с 1; $S[1] \neq S[10]$	1	$[8, 10)$
10	$i \geq R$	$S[0] \neq S[10]$	0	$[8, 10)$

Шаг 9: известное совпадение доходит до R, дальше строка не изучена — сравниваем начиная с $S[R]$. Здесь оно сразу обрывается; могло бы и продолжиться.

Итого 16 сравнений на 11 символов. Обещаем не больше $2n - 2 = 20$.

Доказательство: инвариант и три случая

Инвариант перед шагом i: $Z[1..i)$ верны; $[L, R)$ — Z-блок с самым правым концом среди $1 \le t < i$: $S[L..R) = S[0..R-L)$, $R = \max_t(t + Z[t])$.

Случай 1, $i \ge R$. Ничего не известно, сравниваем $S[i..)$ с $S[0..)$ до различия. Ответ точный по определению.

Случай 2, $i < R$, $j = i - L$. Блок даёт $S[i..R) = S[j..R-L)$, а $Z[j]$ уже известен ($1 \le j < i$).

2а, $Z[j] < R - i$: совпадение $S[j..j+Z[j]) = S[0..Z[j])$ и различие $S[j+Z[j]] \ne S[Z[j]]$ лежат внутри $[j, R-L)$, значит, переносятся на i: $Z[i] = Z[j]$ точно.

2б, $Z[j] \ge R - i$: известно $S[i..R) = S[0..R-i)$, а про $S[R]$ ничего. Сравниваем с $S[R-i]$ дальше — ответ точный.

После шага: если $i + Z[i] > R$, новый блок правее — инвариант сохранён.

Почему всего $O(n)$ сравнений

Каждое сравнение либо **успешно**, либо **неудачно**.

Неудачное сравнение заканчивает шаг: их не больше одного на шаг, всего $\leq n - 1$.

Успешное сравнение на шаге i сравнивает $S[i + Z[i]]$ при $i + Z[i] \geq R$ — то есть символ *правее всех уже изученных*. После шага R становится $i + Z[i]$. R никогда не убывает, поэтому каждый символ строки бывает «новым» один раз: $\leq n - 1$.

Итого $\leq 2n - 2$ сравнений. В трассе: 16 при $n = 11$.

Поиск через $P\#T$: время и память $O(n + m)$.

Gusfield, теорема 1.4.1.

Настоящий код: Z-функция из KACTL

```
11 vi Z(const string& S) {
12   vi z(sz(S));
13   int l = -1, r = -1;
14   rep(i,1,sz(S)) {
15     z[i] = i >= r ? 0 : min(r - i, z[i - l]);
16     while (i + z[i] < sz(S) && S[i + z[i]] ==
       S[z[i]])
17       z[i]++;
18     if (i + z[i] > r)
19       l = i, r = i + z[i];
20   }
21   return z;
22 }
```

- **KACTL** — справочник кода команды KTH для ICPC; короткие проверенные реализации, CC0.
- `vi` = `vector<int>`, `sz` = размер, `rep(i,1,n)` — цикл $1 \le i < n$.
- Строка 13: `l,r` — наши L, R. Старт -1 безопасен: при $i \ge r$ массив не читается.
- Строки 15–19 — наш псевдокод построчно; `z[0]=0`.

kth-competitive-programming/kactl · CC0 content/strings/Zfunc.h:11–22

Откуда мы знаем, что код верен

дополнительно

Инвариант и три случая два кадра назад доказывают *алгоритм*. Код — отдельный вопрос: его сверяем с определением Z на *всех* коротких строках над алфавитом $\{\mathsf{a}, \mathsf{b}\}$.

Реализация	Полный перебор	Расхождений
Наша модель (demo/models.py)	255 строк: длины 0 … 7	0
KACTL и AtCoder	2 047 строк: длины 0 … 10	0

Эталон — наивное сравнение каждой позиции с началом. Для модели проверено и число сравнений: не больше $2n - 2$.

Перебор ловит ошибки кода на коротких входах; доказательство — на любых.

demo/check_models.py; demo/check_published.cpp; numbers.md, N05–N06.

Можно ли читать текст по одному символу и никогда не возвращаться?

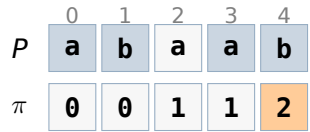
Z-функции нужна склейка $P\#T$ целиком. Хочется читать T потоком: символ пришёл — обработали — забыли.

Вся информация должна быть подготовлена по одному образцу P.

Knuth, Morris, Pratt, *Fast pattern matching in strings*, SIAM J. Comput. 1977; CLRS, 2-е изд., §32.4.

Граница: префикс, который является суффиксом

$\pi[i]$ — длина наибольшего *собственного* префикса $P[0..i]$, который одновременно его суффикс.

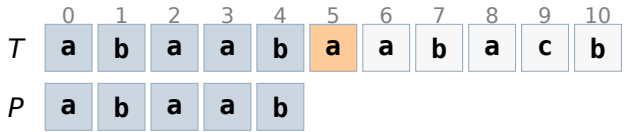


$\pi[4] = 2$: у abaab префикс ab равен суффиксу ab. $\pi[2] = 1$: у aba — a. $\pi[3] = 1$: у abaa — a. $\pi[1] = 0$: у ab границы нет.

«Собственный» значит короче всей строки: иначе $\pi[i] = i + 1$ всегда.

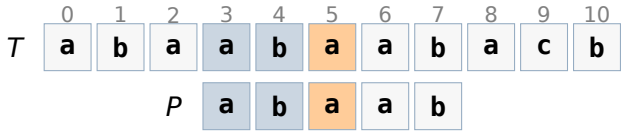
Вопрос залу

Образец abaab найден в позиции 0. Пришёл $T[5] = \text{a}$. С чем его сравнивать?



Возвращаться к $T[1]$ и начинать заново нельзя: текст — поток. Что мы знаем о последних прочитанных символах?

Ответ: продолжаем с границы



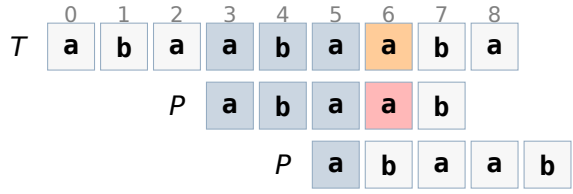
Прочитанный текст кончается на abaab. Его суффикс ab — граница P длины $\pi[4] = 2$, то есть он же префикс P. Значит, $P[0..2)$ уже совпало с $T[3..5)$, и $T[5]$ сравниваем с $P[2]$. Совпало — длина 3, и так далее.

Состояние q — длина наибольшего префикса P, совпадающего с концом прочитанного текста. На каждый символ: пока $P[q] \neq c$, переходим $q \leftarrow \pi[q-1]$; совпало — $q \leftarrow q+1$.

Почему кандидаты — ровно цепочка границ

Пусть совпало q символов: $P[0..q) = T[i-q..i)$, а $T[i] \neq P[q]$.

Новый кандидат длины $q' < q$ должен удовлетворять $P[0..q') = T[i-q'..i)$. Но $T[i-q'..i)$ — это суффикс $P[0..q)$. Значит, $P[0..q')$ — и префикс, и суффикс $P[0..q)$: **граница**. Самый длинный такой — $\pi[q-1]$.



Если и $\pi[q-1]$ не подошло — следующая граница $\pi[\pi[q-1]-1]$, и так до нуля. Более длинный кандидат пропустить нельзя: он тоже обязан быть границей.

Поиск КМП: все вхождения, включая перекрытия

```
1 q = 0
2 for i = 0 .. n−1:
3     while q > 0 and P[q] != T[i]:
4         q = pi[q−1]
5     if P[q] == T[i]:
6         q += 1
7     if q == m:
8         emit(i−m+1)
9         q = pi[m−1]
```

- Строки 3–4: цепочка границ, пока символ не подходит.
- Строки 5–6: совпало — состояние выросло на 1.
- Строки 7–9: $q = m$ — вхождение в $i - m + 1$; продолжаем с границы $\pi[m-1]$, чтобы не потерять перекрытие.
- Текст читается ровно один раз, i не уменьшается.

Учебный псевдокод; индексы с нуля; $m > 0$. CLRS, KMP-Matcher.

Трасса поиска: abaab в abaabaabacb

i	$T[i]$	переходы q	q после	вхождение
0–3	a b a a	совпадения	1, 2, 3, 4	
4	b	$4 \to 5$	$\pi[4] = 2$	**0**
5	a	$P[2] =$ a: $2 \to 3$	3	
6	a	$P[3] =$ a: $3 \to 4$	4	
7	b	$4 \to 5$	$\pi[4] = 2$	**3**
8	a	$P[2] =$ a: $2 \to 3$	3	
9	c	$P[3] \neq$ c: $3 \to \pi[2] = 1$; $P[1] \neq$ c: $1 \to \pi[0] = 0$; $P[0] \neq$ c	0	
10	b	$P[0] \neq$ b	0	

Шаг 9: один символ, два перехода по цепочке. Шаги 4 и 7: вхождение, и состояние не обнуляется, а переходит на границу 2 — поэтому вхождение 3 не потеряно.

Префикс-функция строится тем же переходом

```
1 pi[0] = 0
2 for i = 1 .. m−1:
3     q = pi[i−1]
4     while q > 0 and P[q] != P[i]:
5         q = pi[q−1]
6     if P[q] == P[i]:
7         q += 1
8     pi[i] = q
```

Для abaab:

i	$P[i]$	переходы	$\pi[i]$
1	b	0: $P[0] \neq$ b	0
2	a	0: $P[0] =$ a	1
3	a	1: $P[1] \neq$ a; 0: $P[0] =$ a	1
4	b	1: $P[1] =$ b	2

- Строка 3: берём границу предыдущего префикса и пробуем продлить её символом $P[i]$.
- Кандидаты снова идут по цепочке π: граница без последнего символа — граница более короткого префикса.

Учебный псевдокод; CLRS, Compute-Prefix-Function. В лекции 2010 года та же таблица строилась из Z-функции образца.

Линейность КМП — амортизированная

За один символ q растёт не больше чем на 1: всего увеличений $\leq n$.

Каждый переход $q \leftarrow \pi[q-1]$ строго уменьшает q, а $q \geq 0$: уменьшений не больше, чем было увеличений.

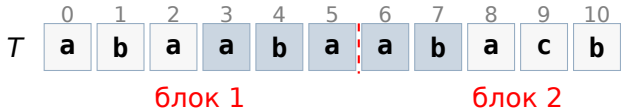
Переходов $\leq n$, сравнений $\leq 2n$. Поиск $O(n)$, подготовка $O(m)$ тем же рассуждением.

Память: π и состояние q — $O(m)$. Самого текста хранить не нужно.

Но: отдельный символ может вызвать длинную цепочку переходов. «Линейно в сумме» не значит «$O(1)$ на символ» — для real-time нужен автомат.

CLRS, §32.4, амортизационный анализ; Knuth–Morris–Pratt 1977.

Текст приходит кусками



Сетевой пакет, буфер файла, страницы памяти: текст не лежит в одном массиве.

Вхождение 3 начинается в первом блоке и кончается во втором. КМП достаточно перенести состояние $q = 3$ в следующий блок и продолжить с его первого байта.

Z-функции здесь нечего предложить: ей нужна склейка целиком.

Настоящий код: КМП в ядре Linux, состояние живёт между блоками

```
42 static unsigned int kmp_find(struct ts_config
       *conf, struct ts_state *state)
43 {
44   struct ts_kmp *kmp = ts_config_priv(conf);
45   unsigned int i, q = 0, text_len, consumed =
       state->offset;
46   const u8 *text;
47   const int icase = conf->flags & TS_IGNORECASE;
48
49   for (;;) {
50     text_len = conf->get_next_block(consumed,
       &text, conf, state);
51
52     if (unlikely(text_len == 0))
53       break;
```

- `lib/ts_kmp.c` — один из трёх движков подсистемы *textsearch* ядра (КМП, Бойер–Мур, naive); её использует netfilter.
- Строка 45: `q` объявлен *до* цикла по блокам — это наше состояние.
- Строка 50: `get_next_block` выдаёт следующий кусок текста, например фрагмент пакета.

torvalds/linux v6.12 · GPL-2.0 lib/ts_kmp.c:42–54. Автор Thomas Graf, 2005.

Linux: внутренний цикл — наш псевдокод построчно

дополнительно

```
55    for (i = 0; i < text_len; i++) {
56      while (q > 0 && kmp->pattern[q]
57          != (icase ? toupper(text[i]) : text[i]))
58        q = kmp->prefix_tbl[q - 1];
59      if (kmp->pattern[q]
60          == (icase ? toupper(text[i]) : text[i]))
61        q++;
62      if (unlikely(q == kmp->pattern_len)) {
63        state->offset = consumed + i + 1;
64        return state->offset - kmp->pattern_len;
65      }
66    }
67
68    consumed += text_len;
69  }
```

torvalds/linux v6.12 · GPL-2.0 lib/ts_kmp.c:55–69

- Строки 56–58: цепочка границ; `prefix_tbl` — наш π.
- Строки 59–61: совпало — `q++`.
- Строки 62–65: возвращается *первое* вхождение; `offset` запоминает, где остановились. `icase` — поиск без учёта регистра.

Z и π: одна информация, разные удобства

	Z-функция	КМП с π
Что подготовлено	совпадения каждой позиции с началом	границы префиксов образца
Как ищем	склейка $P\#T$	поток символов T
Память	$O(n+m)$	$O(m)$
Где ещё полезно	периоды, Бойер–Мур (сейчас)	потоки, автоматы, Ахо–Корасик

Обе таблицы считаются за $O(m)$ и выражаются друг через друга. Z понадобится через несколько кадров: таблица Бойера–Мура — это Z перевёрнутого образца.

Что даёт сравнение справа налево?

КМП читает каждый символ текста. Бойер и Мур (1977) заметили: если сравнивать окно *с конца*, то при неудаче известны сразу две вещи —

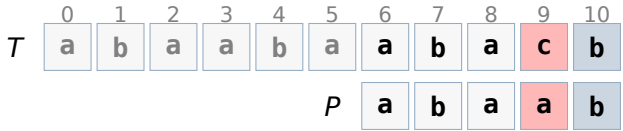
- какой символ текста не подошёл (и где он есть в образце), и
- какой суффикс образца уже совпал.

Каждое знание даёт безопасный сдвиг; берём больший. На обычном тексте многие символы вообще не читаются.

Boyer, Moore, *A fast string searching algorithm*, CACM 1977. Gusfield, гл. 2.2.

Вопрос залу

Окно в позиции 6. Справа налево: b=b, затем c≠a. Куда сдвигать?



Какие начала окна $s > 6$ ещё могут дать вхождение? Подумайте про символ c.

Ответ: два правила, берём больший сдвиг

Плохой символ. c в образце нет: любое окно, накрывающее $T[9]$, невозможно. Начало должно быть ≥ 10: сдвиг 4.

Хороший суффикс. Совпавший b в новом окне должен снова лечь на b образца, причём перед ним не должно стоять a (мы уже знаем, что там не a). В abaab другой такой b нет, и ни одна граница образца не является суффиксом b: сдвиг 5, на всю длину.

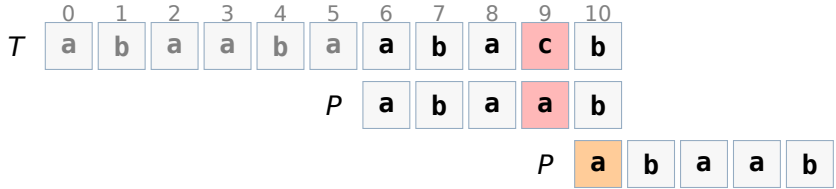
Сдвиг $\max(4, 5) = 5$: окно уходит за конец текста, поиск окончен. Из пяти символов окна прочитаны два.

Правило плохого символа

$\mathrm{last}(c)$ — последняя позиция символа c в образце; -1, если его нет. Для abaab: $\mathrm{last}(\mathsf{a}) = 3$, $\mathrm{last}(\mathsf{b}) = 4$.

Различие в позиции j образца на символе $c = T[s + j]$. Совместим c с его последним вхождением в образец:

$$d_{\mathsf{bad}} = \max\bigl(1,\ j - \mathrm{last}(c)\bigr).$$



Здесь $j = 3$, $\mathrm{last}(\mathsf{c}) = -1$: сдвиг 4. Если $\mathrm{last}(c) > j$, формула даёт 1 — правило бессильно, спасает хороший суффикс.

Правило хорошего суффикса

Совпал суффикс $U = P[j+1..m)$, различие в $P[j]$. Куда можно сдвинуть образец?

1. К другой копии U в P, перед которой стоит *не* $P[j]$ (иначе различие повторится).
2. Нет копии — к самому длинному суффиксу U, который является префиксом P (граница).
3. Нет и его — сдвиг на m.

j	совпавший U	что нашли	сдвиг $\mathrm{good}[j]$
4	пусто	—	1
3	b	другой b есть, но перед ним a — не годится; границ нет	5
2	ab	копия ab в начале образца (перед ней ничего)	3
1	aab	копии нет; граница ab	3
0	baab	копии нет; граница ab	3

Поиск Бойера–Мура и трасса на примере

```
s = 0
while s <= n−m:
    j = m−1
    while j >= 0 and P[j] == T[s+j]:
        j −= 1
    if j < 0:
        emit(s)
        s += full_match_shift
    else:
        bad = max(1, j−last(T[s+j]))
        s += max(bad, good[j])
```

s	сравнений	bad	good	сдвиг
0	5, совпало	—	3	3
3	5, совпало	—	3	3
6	2, $j = 3$	4	5	5

Вхождения 0 и 3. Всего 12 сравнений;

окна 1, 2, 4, 5 не рассматривались вовсе.

Учебный псевдокод; индексы с нуля. Трасса: demo/traces.py.

Таблица N: Z-функция перевёрнутого образца

Чтобы заполнить good, нужно знать для каждого i: сколько символов конец $P[0..i]$ совпадает с концом всего P. Это $N[i]$.

Перевернём образец: $Q =$ baaba. Суффиксы P стали префиксами Q, и $N[i] = Z_Q[m-1-i]$; $N[m-1] = m$ по определению.

i	0	1	2	3	4
$P[i]$	a	b	a	a	b
Z_Q (для baaba)	0	0	0	2	0
$N[i]$	0	2	0	0	5

$N[1] = 2$: ab — конец $P[0..1]$ и конец P. Строка таблицы good для j: нужно i с $N[i] = m-1-j$ (копия суффикса), иначе — граница ($N[i] = i+1$).

Настоящий код: таблицы Бойера–Мура в SMART

```
36 void suffixes(unsigned char *x, int m, int *suff) {
37    int f, g, i;
38    suff[m - 1] = m;
39    g = m - 1;
40    for (i = m - 2; i >= 0; --i) {
41       if (i > g && suff[i + m - 1 - f] < i - g)
42          suff[i] = suff[i + m - 1 - f];
43       else {
44          if (i < g)
45             g = i;
46          f = i;
47          while (g >= 0 && x[g] == x[g + m - 1 - f])
48             --g;
49          suff[i] = f - g;
50       }
51    }
52 }
```

- **SMART** — String Matching Algorithms Research Tool: сотня реализаций поиска подстроки с одним интерфейсом (Фаро, Лекрок, 2010–); на нём меряют новые алгоритмы.
- `suff` — наш N; это Z-алгоритм справа налево: `f`,`g` — зеркальные L, R. Строки 42–43 копируют ответ из блока, 44–50 досравнивают.

smart-tool/smart · GPL-3.0 source/algos/bm.c:36–52. Faro, Lecroq, *The exact online string matching problem: a review*, ACM CSUR 2013.

SMART: из N в сдвиги

дополнительно

```
54 void preBmGs(unsigned char *x, int m, int bmGs[]) {
55    int i, j, suff[XSIZE];
56    suffixes(x, m, suff);
57    for (i = 0; i < m; ++i) bmGs[i] = m;
58    j = 0;
59    for (i = m - 1; i >= 0; --i)
60       if (suff[i] == i + 1)
61          for (; j < m - 1 - i; ++j)
62             if (bmGs[j] == m)
63                bmGs[j] = m - 1 - i;
64    for (i = 0; i <= m - 2; ++i)
65       bmGs[m - 1 - suff[i]] = m - 1 - i;
66 }
```

- Строка 57: по умолчанию сдвиг m (пункт 3 правила).
- Строки 59–63: `suff[i]==i+1` — префикс $P[0..i]$ является суффиксом P, то есть граница; ею закрываются ещё не заполненные j (пункт 2).
- Строки 64–65: внутренняя копия суффикса длины `suff[i]`, кончающаяся в i (пункт 1); более поздние i перезаписывают — остаётся ближайшая копия.

smart-tool/smart · GPL-3.0 source/algos/bm.c:54–66

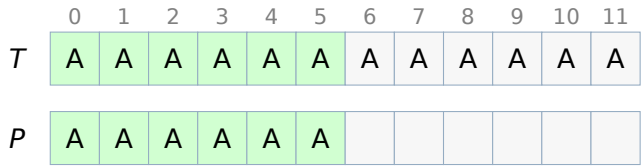
Настоящий код: Бойер–Мур в ядре Linux

```
92    while (shift < text_len) {
93      DEBUGP("Searching in position %d (%c)\n",
94             shift, text[shift]);
95
96      i = matchpat(&bm->pattern[bm->patlen-1],
      bm->patlen,
97             &text[shift], icase);
98      if (i == bm->patlen) {
99        /* London calling... */
100       DEBUGP("found!\n");
101       return consumed + (shift-(bm->patlen-1));
102     }
103
104     bs = bm->bad_shift[text[shift-i]];
105
106     /* Now jumping to... */
107     shift = max_t(int, shift-i+bs,
      shift+bm->good_shift[i]);
```

- `shift` — *правый* край окна, $s + m - 1$; `i` — сколько совпало справа, наш $m - 1 - j$. Строка 96: `matchpat` сравнивает справа налево.
- Строка 107: максимум из двух правил; `bad_shift` хранит расстояния, `good_shift[i]` — наш good.
- В заголовке файла автор предупреждает: вхождение, разрезанное границей блоков, этот код пропустит.

torvalds/linux v6.12 · GPL-2.0 lib/ts_bm.c:92–107. Автор Pablo Neira Ayuso, 2005.

Бойер–Мур тоже повторяет сравнения



Ищем все вхождения a^m в a^n: после каждого совпадения сдвиг на 1 (граница длины $m-1$), и все m символов сравниваются заново: $\Theta(nm)$.

Окно знает про текст только то, что прочитало само. Что, если запомнить?

Gusfield, §2.2.6; Galil, 1979 — первое исправление для случая после полного совпадения.

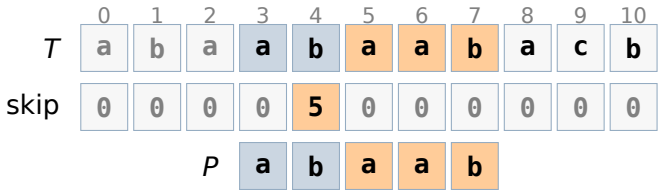
Можно ли запомнить, что уже сравнили в предыдущих окнах?

Апостолико и Джанкарло (1986): оставить сдвиги Бойера–Мура, но завести память о тексте — массив skip: «в этой позиции текста заканчивается совпадение с k последними символами образца».

Тогда следующее окно, дойдя при сравнении справа налево до такой позиции, перепрыгивает k символов, не читая их.

Apostolico, Giancarlo, *The Boyer–Moore–Galil string searching strategies revisited*, SIAM J. Comput. 1986. В плане Макарова — неделя 9.

Как память помогает на нашем примере



Окно 0 совпало целиком: записали $\mathrm{skip}[4] = 5$ — «в $T[4]$ кончается совпадение с 5 последними символами P», то есть $T[0..5) = P$.

Окно 3: сравнили $T[7], T[6], T[5]$ с $P[4], P[3], P[2]$ (три чтения). Дошли до $T[4]$: память говорит $k = 5$, а таблица $N[1] = 2$ говорит: $P[0..2)$ — конец образца. Значит $T[3..5) = P[3..5) = P[0..2)$: вхождение найдено **без двух последних сравнений**.

Три случая на одной позиции

Сканируем окно справа налево, стоим в позиции i образца. Известно $k = \mathrm{skip}$ для этой позиции текста и $h = N[i]$ — сколько символов конец $P[0..i]$ совпадает с концом P.

Случай	Что известно	Что делаем
$k = 0$	ничего	сравниваем $P[i]$ и $T[s + i]$, как в BM
$k < h$	k символов совпали; перед ними в тексте стоит *не* суффиксный символ, а в образце — он	пропускаем k; дальше — известное различие
$k = h$	k символов совпали; что перед ними — неизвестно	пропускаем k; сравниваем дальше
$k > h$	h символов совпали; перед ними в образце *не* суффиксный символ, а в тексте — он	пропускаем h; известное различие, либо $h = i + 1$ — весь остаток совпал

В примере: $k = 5 > h = 2 = i + 1$ — последняя строка, вхождение.

Настоящий код: AG в SMART

```
82      i = m - 1;
83      while (i >= 0) {
84          k = skip[i];
85          s = suff[i];
86          if (k > 0)
87              if (k > s) {
88                 if (i + 1 == s)
89                    i = (-1);
90                 else i -= s;
91                 break;
92              }
93              else {
94                  i -= k;
95                  if (k < s) break;
96              }
97          else {
98              if (x[i] == y[i + j]) --i;
99              else break;
100         }
```

smart-tool/smart · GPL-3.0 source/algos/ag.c:82–100

- Строки 84–85: k=skip[i], s=suff[i] — наши k и h.
- Строки 87–92: $k > h$ — последняя строка таблицы; 93–96: $k \leq h$ — пропуск и либо различие, либо продолжение.
- Строки 97–99: $k = 0$ — обычное сравнение.
- Строки 108–113 (не показаны): после сдвига массив skip физически копируется влево через memcpy — а диапазоны перекрываются.

Откуда мы знаем: линейное число сравнений — не линейное время

Теорема: AG делает не больше $2n$ сравнений символов. Доказательство считает *сравнения*.

Но реализация после каждого окна сдвигает массив skip из m чисел: на a^n сдвиг равен 1, и каждое окно копирует почти m чисел — $\Theta(nm)$ операций, которых теорема не видит.

Лечится кольцевым буфером или хранением по абсолютным позициям текста. В замере ниже оставлен код SMART как есть — и он проигрывает.

Доказательство верно ровно для той работы, которую считает. Тот же урок, что переполнение в бинарном поиске из лекции 2.

Crochemore et al., *Speeding up two string-matching algorithms*, Algorithmica 1994 — граница $1{,}5n$ сравнений.

Гарантированный худший случай тоже имеет цену

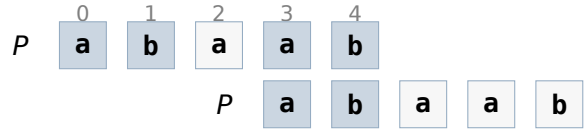
Разработчики CPython взяли не один из четырёх, а пятый алгоритм — **Two-Way** Крочемора и Перрена (1991): линейное время, *постоянная* память, без таблиц по алфавиту. Он же в glibc (`memmem`, `strstr`) и в Rust.

Шаг	Что выяснили
Сообщение и воспроизводитель	один байт меняет сдвиг окна
Разбор фильтра	это настоящий B, не коллизия маски
Первые варианты Two-Way	часть коротких поисков замедлилась
Итоговый PR #22904 (Dennis Sweeney)	пороги + адаптивное переключение
Python 3.10, октябрь 2021	гибрид в выпуске

обсуждение #86138; PR #22904; Crochemore, Perrin, *Two-way string-matching*, J. ACM 1991.

Период строки

p — **период** строки P, если $P[i] = P[i+p]$ при всех $0 \le i < m - p$: строка совпадает сама с собой, сдвинутой на p. Наименьший период обозначаем $p(P)$.

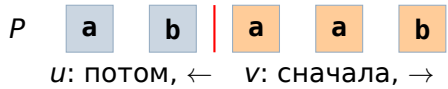


У abaab период 3: сдвинутая копия совпадает на ab. Период и граница — одно и то же знание: $p(P) = m - \pi[m-1]$ (граница ab длины 2, $5 - 2 = 3$).

Примеры: ABABAB — период 2 (повтор может быть неполным: ABABA тоже 2); GCAGAGAG — период 7 (совпадает только последняя G); строка без границы — период m.

Two-Way: разрезать образец и сравнивать в две стороны

Образец режется на две части $P = u \mid v$ в **критической** позиции ℓ.
Для abaab: $\ell = 2$, $u =$ ab, $v =$ aab.



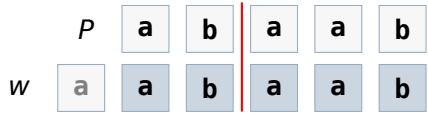
В окне s: сначала сравниваем v слева направо, начиная от разреза. Если v совпала целиком — сравниваем u справа налево.

Два правила сдвига:

1. Различие в v на расстоянии i от разреза — сдвиг на $i + 1$.
2. v совпала, различие в u или полное совпадение — сдвиг на **период** p образца; при этом известно, что первые $m - p$ символов нового окна уже совпали (`memory`).

Почему разрез должен быть критическим

Период можно мерить и *в одной точке*. **Локальный период** в позиции ℓ — длина кратчайшего w, которое стоит и слева, и справа от разреза (у краёв строки w может выступать наружу).



Для ab | aab: $w = \text{aab}$ (слева выступает за край); b и ab не подходят. Локальный период 3 — равен периоду строки.

Теорема о критической факторизации (Cesari, Vincent, 1978): есть разрез, где они равны — *критический*; Крочемор и Перрен находят его за $O(m)$.

Crochemore, Perrin 1991; разбор для CPython — stringlib_find_two_way_notes.txt рядом с кодом; Lothaire, *Algebraic Combinatorics on Words*, гл. 8.

Трасса Two-Way: abaab в abaabaabacb

$u = \mathrm{ab}$, $v = \mathrm{aab}$, период $p = 3$, образец периодический: $u = P[3..5)$.

s	memory	v слева направо	u справа налево	сдвиг
0	0	aab = $T[2..5)$	ba = $T[1], T[0]$ — **вхождение 0**	$p = 3$, memory $= 2$
3	2	aab = $T[5..8)$	первые 2 уже известны — **вхождение 3**	$p = 3$, memory $= 2$
6	2	$T[8] = \mathrm{a}$, $T[9] = \mathrm{c} \neq \mathrm{a}$: $i = 1$	—	$i + 1 = 2$, memory $= 0$

10 сравнений — как у AG, но без массива skip и без таблиц: память $O(1)$.

Чтение символов текста не возвращается дальше, чем на m назад; каждый символ сравнивается не больше двух раз: $\leq 2n$ сравнений.

demo/twoway_trace.py. Crochemore, Perrin 1991, теорема 3.

Настоящий код: CPython проверяет периодичность

```
313     p->cut = STRINGLIB(_factorize)(needle,
        len_needle, &(p->period));
314     assert(p->period + p->cut <= len_needle);
315     p->is_periodic = (0 == memcmp(needle,
316                                   needle +
        p->period,
317                                   p->cut *
        STRINGLIB_SIZEOF_CHAR));
318     if (p->is_periodic) {
319         assert(p->cut <= len_needle/2);
320         assert(p->cut < p->period);
321     }
322     else {
323         // A lower bound on the period
324         p->period = Py_MAX(p->cut, len_needle -
        p->cut) + 1;
325     }
```

- Строка 313: `_factorize` — два максимальных суффикса, более поздний разрез; p здесь структура подготовки, не образец.
- Строки 315–317: периодичен ли образец — совпадает ли *u* с образцом, сдвинутым на период. Для abaab: ab = ab, да.
- Строки 322–324: иначе период неизвестен, берётся нижняя оценка $\max(\ell, m - \ell) + 1$ — ею и сдвигаемся.

python/cpython 3.10.0 · PSF Objects/stringlib/fastsearch.h:313–325

Вопрос залу

Two-Way линеен и не требует памяти. Почему бы не включить его всегда?

Подсказка: вспомните формулу окупаемости фильтра. Что здесь «фильтр», а что «точная проверка»?

Ответ: подготовка Two-Way стоит $O(m)$

Старый цикл — дешёвый фильтр: на обычных данных почти ничего не читает. Two-Way сначала платит $O(m)$ за разрез; при коротком тексте это дороже поиска. Нужна **политика**: когда платить.

```
554     if (mode != FAST_RSEARCH) {
555         if (m >= 100 && w >= 2000 && w / m >= 5) {
556             /* For larger problems where the needle isn't a
        huge
557                percentage of the size of the haystack, the
        relatively
558                expensive O(m) startup cost of the two-way
        algorithm
559                will surely pay off. */
560             if (mode == FAST_SEARCH) {
561                 return STRINGLIB(_two_way_find)(s, n, p, m);
562             }
563             else {
564                 return STRINGLIB(_two_way_count)(s, n, p,
        m, maxcount);
565             }
```

- Строка 554: длинный образец, много позиций ($w = n - m$), текст заметно длиннее — сразу Two-Way. Пороги 100, 2000, 5 подобраны по замерам.

python/cpython 3.10.0 · PSF fastsearch.h:554–565

Вторая ветка: начать дёшево и считать потраченное

```
609                 /* miss: check if next
    character is part of pattern */
610                 if (!STRINGLIB_BLOOM(mask,
    ss[i+1])) {
611                     i = i + m;
612                 }
613                 else {
614                     i = i + skip;
615                 }
616                 hits += j + 1;
617                 if (hits >= m / 4 && i < w -
    1000) {
618                     /* We've done O(m)
    fruitless comparisons
619                        anyway, so spend the
    O(m) cost on the
620                        setup for the two-way
    algorithm. */
```

python/cpython 3.10.0 · PSF fastsearch.h:609–620

- Строка 616: `hits` копит число *бесплодных* сравнений.
- Строка 617: набралось $m/4$ — работа, сравнимая с подготовкой, уже потрачена; переключаемся на Two-Way, если впереди достаточно текста.
- Фильтр не выключили — за ним следят: формула окупаемости стала счётчиком.

perf: исчезла работа, а не промахи предсказателя

На один вызов, приближённо	3.9.0	3.10.0
Инструкции	192 979 197	1 774 559
Ветвления	64 208 029	418 102
Такты	62 789 048	413 599

Промахи предсказателя у обоих — не более $0{,}01\%$. Ветвления были *предсказуемыми*: процессор исполнял их быстро, просто их было в 150 раз больше.

В лекции 1 мы видели обратное: столько же работы, но промахи. Счётчики различают эти два случая; секундомер — нет.

Наш запуск, CPU 1, `perf stat -r5`: cycles, instructions, branches, branch-misses; demo/perf-cpython39.txt, perf-cpython310.txt; numbers.md, N02.

Что сделал программист

1. Получил воспроизводитель; сохранил тот же ответ.
2. Нашёл ветку, которая делает лишнюю работу, — и убедился, что фильтр ни при чём.
3. Добавил алгоритм с гарантией худшего случая.
4. Измерил не только плохой вход, но и обычные — и нашёл регрессии.
5. Выбрал, *когда* за гарантию платить: пороги и счётчик.

Та же дилемма повторится сейчас с очень дешёвым фильтром — SIMD.

Несколько кандидатов за одну инструкцию

Можно ли проверить сразу 32 начала окна?

SIMD (single instruction, multiple data) — одна инструкция обрабатывает вектор из многих байтов: 32 сравнения за раз.

AVX2 — конкретный набор таких инструкций x86 с 256-битными регистрами (32 байта); SSE2 — 16 байт, AVX-512 — 64, NEON на ARM — 16. Наша модель написана прямо на AVX2; библиотеки пишут код над абстрактным вектором.

Идея фильтра: выбрать в образце два байта и векторно проверить их сразу во многих позициях. Кто прошёл — проверяется точно, `memcmp`.

Денис Ярошевский, *SIMD Substring in a String*, C++Online 2025: запись; Wojciech Muła, SIMD-friendly algorithms for substring searching.

Два байта — необходимое условие, маска — кандидаты

Позиции a, b в образце: вхождение в s возможно, только если $T[s+a] = P[a]$ и $T[s+b] = P[b]$.

	0	1	2	3	4	5	6	7
$T[s]$	a	b	a	c	a	b	a	d
$T[s+1]$	b	a	c	a	b	a	d	x
$= a$	1	0	1	0	1	0	1	0
$= b$	1	0	0	0	1	0	0	0
AND	1	0	0	0	1	0	0	0

Для образца ab ($a = 0$, $b = 1$) остаются начала 0 и 4; для длинного образца это ещё кандидаты. Две загрузки, два сравнения, одно И — маска кандидатов; по её единицам — точная проверка.

Наша AVX2-модель: 32 начала за итерацию

```
35 __attribute__((target("avx2")))
36 int pair_filter(unsigned char* p,int m,unsigned char* t,int
       n) {
37     if(m==0) return n+1;
38     if(m>n) return 0;
39     int i=0,count=0,windows=n-m+1;
40     auto a=_mm256_set1_epi8(p[0]), b=_mm256_set1_epi8(p[m-1]);
41     for(; i+32<=windows; i+=32) {
42         auto x=_mm256_loadu_si256((const __m256i*)(t+i));
43         auto y=_mm256_loadu_si256((const __m256i*)(t+i+m-1));
44         unsigned mask=_mm256_movemask_epi8(_mm256_and_si256(
45             _mm256_cmpeq_epi8(x,a),_mm256_cmpeq_epi8(y,b)));
46         while(mask) {
47             int offset=__builtin_ctz(mask);
48             count+=std::memcmp(t+i+offset,p,m)==0;
49             mask&=mask-1;
50         }
51     }
52     for(;i<windows;++i) count+=std::memcmp(t+i,p,m)==0;
53     return count;
54 }
```

- Строка 40: 32 копии первого и последнего байта.
- Строки 42–45: две загрузки со сдвигом $m-1$, два сравнения, И, `movemask` — 32 бита кандидатов.
- Строки 46–50: `ctz` — младшая единица; `memcmp` проверяет окно; `mask&=mask-1` снимает бит.
- Строка 52: хвост — скалярно.

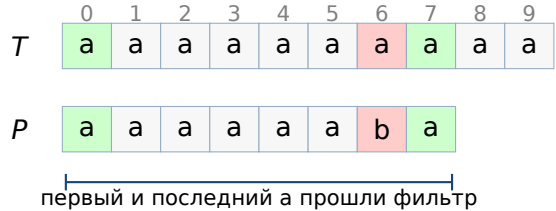
Учебная реализация курса, подготовлена с помощью ИИ; demo/search_bench.cpp:35–54. `target("avx2")`: вызывающий код проверяет поддержку AVX2.

Вопрос залу

Текст — миллион букв a. Образец — $a^{126}ba$. Что сделает фильтр по первому и последнему байту?

Вхождений нет: в тексте нет b. Сколько кандидатов пройдёт фильтр?

Ответ: фильтр пропускает всех — и стоит дороже, чем ничего



Оба выбранных байта — a: все 10^6 позиций — кандидаты. Каждую проверяет `memcmp`, который отвергает её на 127-м байте.

Наш замер на этом входе: **1,908 мс** с фильтром против **1,660 мс** у простого `memcmp` в каждом окне. Фильтр не отсёк никого и добавил свою цену — C_f при $q(1-f) \approx 0$.

demo/search-comparison-results.csv, вход pair_trap; numbers.md, N09.

Что делает настоящая библиотека: Rust memchr

`memchr` — крейт, через который ищут подстроки стандартная библиотека Rust и ripgrep. Автор Andrew Gallant (BurntSushi).

Из README, раздел «Algorithms used»:

> *In all other cases, Two-Way is used. If possible, a prefilter based on the "Generic SIMD" algorithm … is used to find candidates quickly.* ***A dynamic heuristic is used to detect if the prefilter is ineffective, and if so, disables it.***

Точная проверка — Two-Way, как в CPython. Фильтр — пара байтов, как у нас, но пара выбирается по редкости байтов. И за фильтром *следят*: тот же счётчик, что `hits` в CPython.

Код над абстрактным вектором собирается под SSE2, AVX2 и NEON.

memchr 2.7.6, README; код маски: src/arch/generic/packedpair.rs:236–255 (в refs/).

Как сравниваем скорость

- Один контракт для всех: **считаем все вхождения**, включая перекрытия.
- Подготовка, временные массивы и подсчёт входят во время: образец каждый раз новый.
- Семь серий, в каждой движки в случайном порядке; медиана на один проход. Текст уже в памяти.
- Чужой код как есть: SMART (КМП, BM, AG, Two-Way), AtCoder (Z), glibc `memmem`. Наши модели: наивный, `memcmp` в каждом окне, AVX2-пара.
- 32 439 проверок правильности перед замером: все движки дали одинаковые ответы.

CPU 1 · Ryzen AI MAX+ 395 · GCC 13.3 -O3 · glibc 2.39 · demo/run_comparison.py · numbers.md, N09.

Четыре входа — четыре разные нагрузки

Текст	Байт	Образец	Вхождений
Случайные 26 букв	1 048 576	`abracadabrax`	0
Английский словарь (370 105 слов)	4 234 910	`tion`	10 910
Только буква a	1 048 576	a^{128}	1 048 449
Только буква a	1 048 576	a^{126}`ba`	0

Словарь — тот же файл, что в лекции 4. Два последних входа — худшие случаи: совпадение везде и ловушка для фильтра по паре байтов.

Редкие совпадения: побеждает дешёвый фильтр

Медиана, мс (меньше — лучше)	26 букв, 0 вхождений	словарь, `tion`
Наивный посимвольный	0,630	3,191
`memcmp` в каждом окне	1,312	5,310
AtCoder Z: *P#T*	3,253	17,863
SMART КМП	1,667	7,179
SMART Бойер–Мур	0,374	4,703
SMART AG (+`memmove`)	0,584	7,137
SMART Two-Way	0,486	3,511
glibc `memmem`, в цикле	0,054	1,065
Наша AVX2-пара	**0,026**	**0,235**

Сравнивать внутри столбца. Наивный поиск здесь быстрее КМП: различие в первой букве, а у КМП больше работы на символ.

demo/search-comparison-results.csv · 7 серий · numbers.md, N09.

Повторы: побеждает гарантия худшего случая

Медиана, мс (меньше — лучше)	$P = a^{128}$, 1 048 449 вхождений	$P = a^{126}ba$, 0 вхождений
Наивный посимвольный	26,956	26,354
`memcmp` в каждом окне	1,649	1,660
AtCoder Z: *P#T*	3,446	3,429
SMART КМП	1,433	1,436
SMART Бойер–Мур	26,795	1,847
SMART AG (+`memmove`)	9,813	4,725
SMART Two-Way	**1,033**	**0,411**
glibc `memmem`, в цикле	44,304	1,436
Наша AVX2-пара	1,877	1,908

Бойер–Мур на a^{128} — как наивный: после каждого совпадения сравнивает всё заново. AG сравнивает меньше, но двигает массив.

demo/search-comparison-results.csv · 7 серий · numbers.md, N09.

За что платит каждая реализация

- **AVX2-пара:** 20 раз быстрее на редких совпадениях; на ловушке медленнее `memcmp` — фильтр не отсёк никого. Поэтому memchr его выключает.
- **Z через *P#T*:** 17,9 мс на словаре — два временных массива по `int` на байт, 32 MiB. Цена интерфейса, не алгоритма.
- **SMART AG:** линейное число сравнений, но `memmove` массива `skip` на каждом окне — хуже КМП.
- **glibc:** 44 мс — миллион вызовов с подготовкой в каждом; один поиск так не стоит.
- **Two-Way:** ни одного худшего случая и ни одной таблицы — поэтому он в glibc, CPython и Rust.

numbers.md, N09; memchr: отключение бесполезного фильтра.

Что выбрать для своей задачи

Задача	С чего начать
Первое вхождение, текст в памяти	библиотечный `find` / `memmem`: Two-Way + фильтр
Текст приходит потоком; все перекрытия	КМП: $O(n+m)$, память $O(m)$, состояние — одно число
Длинный образец, большой алфавит, редкие совпадения	Бойер–Мур или SIMD-фильтр; проверить замером
Совпадения каждой позиции с началом строки	Z-функция: ответ сразу для всех позиций
Много образцов сразу	Ахо–Корасик — следующий раздел

n — длина текста, m — образца. Если образец повторно используется, подготовку меряют отдельно: здесь она входила в каждый вызов.

Измерения N09 плюс доказанные свойства; это ориентиры, не рейтинг.

А если образцов тысячи?

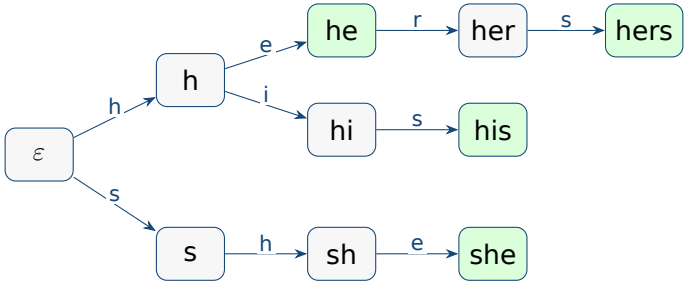
Антивирус, фильтр спама, `grep -F` со списком слов: один текст, много образцов. Запускать КМП для каждого — $O(n \cdot \text{число образцов})$.

Ахо и Корасик (1975): объединить все образцы в одно **trie** (лекция 4) и обобщить КМП: вместо одной границы — суффиксная ссылка в узле. Текст читается один раз.

Образцы: `he`, `she`, `his`, `hers`. Текст: `ushers`. Нужно найти все пары «позиция, номер образца».

Aho, Corasick, *Efficient string matching: an aid to bibliographic search*, CACM 1975.

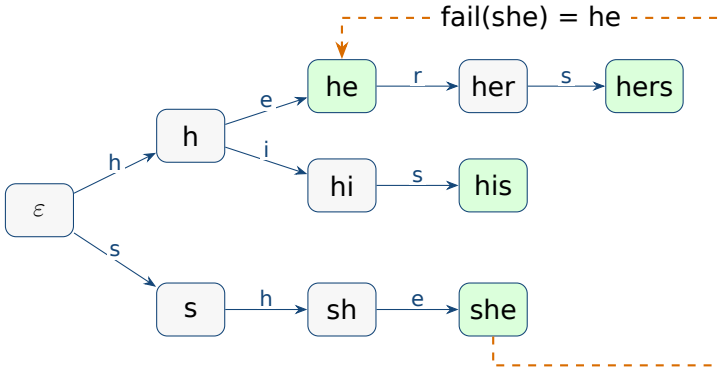
Trie объединяет общие префиксы



В узле — путь от корня; зелёные узлы заканчивают образец.

Состояние после чтения текста — самый длинный суффикс прочитанного, который является путём в trie. После ush это sh: предшествующая u уже не нужна.

Суффиксная ссылка: куда идти после несовпадения



$\mathrm{fail}(v)$ — самый длинный собственный суффикс пути v, который есть в trie.

Для she ссылка ведёт в he; для sh — в h; для he — в корень.

Это π из КМП: там граница была числом, здесь — узлом.

Прочитали ushe. Состояние — узел she. Что выдать?

Терминальный узел один — she. Но образцов четыре.

Ответ: выдать всё по цепочке ссылок

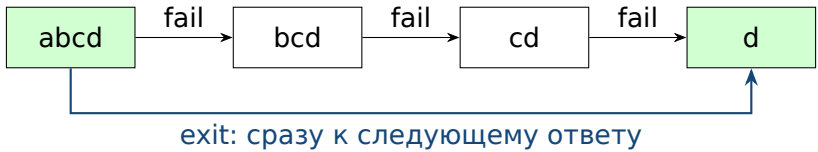
Прочли	Состояние	Выдача (позиция, образец)
u	корень	—
us	s	—
ush	sh	—
ushe	she	(1, she), (2, he)
usher	her	—
ushers	hers	(2, hers)

В состоянии she кончается и he: он — суффикс, то есть лежит на цепочке fail. Один флаг узла потерял бы его.

Шаг r: из she нет ребра r; $\mathrm{fail}(\mathrm{she}) = \mathrm{he}$, оттуда ребро r ведёт в her.

Выходная ссылка: прыгать сразу к следующему ответу

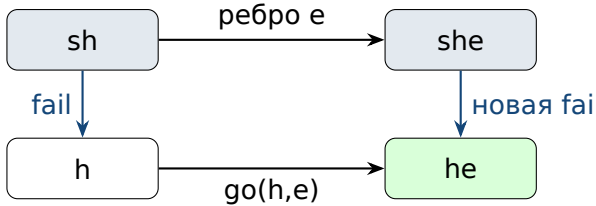
Другой пример: образцы abcd, bcde, cdef, d.



Цепочка fail от abcd: bcd → cd → d. Два промежуточных узла — не образцы; идти по ним на каждом символе дорого.

$\mathrm{exit}(v)$ — ближайший терминальный узел на цепочке fail. От abcd сразу к d. Тогда каждый шаг выдачи даёт ответ: $O(n + \mathrm{occ})$.

Почему ссылки строятся в ширину



Известно $\mathrm{fail}(\mathsf{sh}) = \mathsf{h}$. Из h по букве e попадаем в he. Значит, $\mathrm{fail}(\mathsf{she}) = \mathsf{he}$. В общем виде:

$$\mathrm{fail}(u) = \mathrm{go}(\mathrm{fail}(v), c) \quad \text{для ребра } v \xrightarrow{c} u.$$

Нужны готовые переходы более коротких путей — поэтому обход в ширину, по уровням. Для детей корня $\mathrm{fail} =$ корень отдельно.

Построение и поиск

```
while queue is not empty:
    v = queue.pop_front()
    for c in alphabet:
        if child[v][c] exists:
            u = child[v][c]
            go[v][c] = u
            f = go[fail[v]][c]
            fail[u] = f
            exit[u] = f if output[f] else exit[f]
            queue.push(u)
        else:
            go[v][c] = go[fail[v]][c]
```

```
v = root
for i = 0 .. n−1:
    v = go[v][T[i]]
    u = v
    while u != root:
        for id in output[u]:
            emit(i−length[id]+1, id)
        u = exit[u]
```

- Построение — BFS: строки 5–8 — ссылка через переход родителя, 9 — выходная ссылка; 11 — отсутствующее ребро заменяется готовым переходом: **полный автомат**, один переход на символ.
- Поиск: строка 3 — переход; 4–7 — выдача по цепочке exit.
- Цена: подготовка и память $O(V\sigma + L)$, поиск $O(n + \mathrm{occ})$.

Учебный псевдокод (начало — pseudo/ac_init.txt: корень и его дети). V — узлов, L — сумма длин образцов, σ — алфавит.

Настоящий код: Ахо–Корасик из KACTL, узел и BFS

```
47    queue<int> q;
48    for (q.push(0); !q.empty(); q.pop()) {
49      int n = q.front(), prev = N[n].back;
50      rep(i,0,alpha) {
51        int &ed = N[n].next[i], y = N[prev].next[i];
52        if (ed == -1) ed = y;
53        else {
54          N[ed].back = y;
55          (N[ed].end == -1 ? N[ed].end :
      backp[N[ed].start])
56            = N[y].end;
57          N[ed].nmatches += N[y].nmatches;
58          q.push(ed);
59        }
60      }
61    }
```

- Узел (строки 22–25): `next[26]` — переходы, `back` — fail, `end` — образец узла.
- Строка 52: `y` — переход из fail родителя; 53: пустое ребро заполняется им; 55: иначе это fail ребёнка.
- Строки 56–57: связывают выходы (наш exit).

kth-competitive-programming/kactl · CC0 content/strings/AhoCorasick.h:47–61

KACTL: поиск — один переход на символ

дополнительно

```
63  vi find(string word) {
64    int n = 0;
65    vi res; // ll count = 0;
66    for (char c : word) {
67      n = N[n].next[c - first];
68      res.push_back(N[n].end);
69      // count += N[n].nmatches;
70    }
71    return res;
72  }
73  vector<vi> findAll(vector<string>& pat, string word) {
74    vi r = find(word);
75    vector<vi> res(sz(word));
76    rep(i,0,sz(word)) {
77      int ind = r[i];
78      while (ind != -1) {
79        res[i - sz(pat[ind]) + 1].push_back(ind);
80        ind = backp[ind];
81      }
82    }
83    return res;
84  }
```

- Строка 67: весь поиск — `n = next[c]`. `find` даёт для позиции только *самый длинный* образец: ещё не все совпадения.
- Строки 76–81: `findAll` разворачивает цепочку `backp`: вложенные и повторные образцы; начало $= i - |\text{pat}| + 1$.

Один текст, много разных запросов

Всё, что было сегодня, готовит *образец*: таблицы КМП и Бойера–Мура, разрез Two-Way, автомат Ахо–Корасика. Текст читается заново для каждого запроса.

Что повторяется	Что готовим	Алгоритмы
Один образец, много текстов	образец	КМП, Бойер–Мур, Two-Way
Набор образцов, много текстов	общий автомат	Ахо–Корасик
Один текст, много образцов	**текст**	суффиксное дерево, суффиксный массив

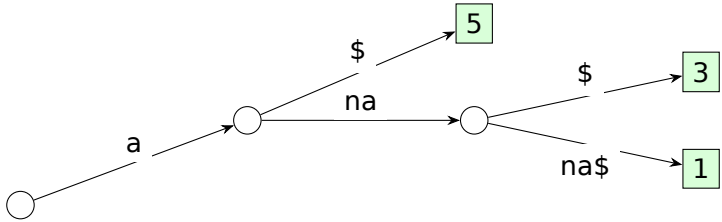
Поиск по геному, по корпусу, по коду: текст один и огромный, запросы приходят потоком. Хочется ответа за $O(m)$, не трогая n.

Подстрока — это префикс суффикса

Начало	Суффикс banana$
0	banana$
1	anana$
2	nana$
3	ana$
4	na$
5	a$
6	$

Образец ana входит в текст ровно там, где он — *префикс* какого-то суффикса: суффиксов 1 и 3.
Положим все суффиксы в trie (лекция 4). Тогда поиск образца — спуск по trie за $O(m)$, а все вхождения — листья под достигнутым узлом.
Терминатор $ не встречается в тексте: все суффиксы различимы и каждый кончается в листе.

Суффиксное дерево: сжатый trie суффиксов



Показана только ветвь суффиксов на a.

Суффиксный trie может иметь $\Theta(n^2)$ узлов: суммарная длина суффиксов $n + (n-1) + \cdots + 1$. Сожмём цепочки без ветвления в одно ребро, метку храним как пару индексов в тексте: $O(n)$ узлов. Поиск $O(m + \mathrm{occ})$. Построение за $O(n)$ — алгоритм Укконена, **следующая лекция**.

Что уносим

- **Фильтр говорит только «нет».** «Возможно» всегда требует точной проверки. Фильтр окупается, когда «нет» частое, а проверка дорогая, — и за этим надо следить (CPython, memchr).
- **Совпавшие части — информация.** Z хранит её про строку, π — про образец, Бойер–Мур — в сдвигах, AG — про текст, Two-Way — в периоде.
- **Один проход по тексту** возможен и для одного образца (КМП), и для тысяч (Ахо–Корасик).
- **Доказательство, чтение кода и замер отвечают на разные вопросы.** AG линеен по сравнениям и проигрывает по времени; naive быстрее КМП на случайном тексте.
- **Когда текст один, готовят текст** — суффиксные структуры, следующая лекция.

Спасибо за внимание

https://github.com/alk-2000/da-lectures